UT3 Bloque 2
Parámetros formularios GET y POST

Desarrollo Web en Entorno Servidor

Cómo viajan los datos del navegador al servlet y vuelven como respuesta

En este bloque aprenderás construyendo peticiones cada vez más completas. Empezaremos escribiendo parámetros en una URL, continuaremos con formularios GET y POST y terminaremos validando en el servidor una inscripción con varios campos. Cada práctica sigue el recorrido completo de los datos.

Punto de partida. Ya conocemos Tomcat, las rutas con @WebServlet, doGet, doPost, request y response. Usaremos Java 17, Tomcat 10.1 y jakarta.servlet. Sustituye ut3-web por el context path real de tu proyecto.

Cómo trabajar con este bloque

Escribe y ejecuta cada ejemplo antes de pasar al siguiente. Después de obtener el resultado, explica en voz alta qué salió del navegador, qué guardó Tomcat en request, qué transformación realizó Java y qué se escribió en response.

Práctica	Producto	Idea que consolida
1	Calculadora GET	Parámetros y conversión a int
2	Saludo validado	Ausente no es lo mismo que vacío
3	Formulario GET	name conecta HTML con Java
4	Formulario de alumno	Controles y conversiones
5	Conversión a POST	Cambia el transporte, no getParameter
6	Alta de producto	Validación completa en servidor
Final	Inscripción	Consulta GET y alta simulada POST

1 Recuperación del Bloque 1

Una petición HTTP no solo indica qué ruta queremos. También puede transportar información. Observa esta dirección:

```
http://localhost:8080/ut3-web/saludo?nombre=Ana&edad=20
```

Parte	Valor	Función
URL	Dirección completa	Permite al navegador localizar servidor, aplicación, ruta y datos.
Context path	/ut3-web	Selecciona la aplicación desplegada.
Ruta	/saludo	Selecciona el servlet mediante @WebServlet.
Query string	nombre=Ana&edad=20	Transporta parámetros después de ?.
Parámetros	nombre y edad	Claves que el servlet consultará.
Valores	Ana y 20	Texto asociado a cada clave.

```
Navegador -- GET /ut3-web/saludo?nombre=Ana&edad=20 --> Tomcat
Tomcat -- doGet(request, response) --> SaludoServlet
request: nombre="Ana", edad="20"
```

No confundas ruta y parámetros. La ruta decide qué servlet recibe la petición. Los parámetros son datos que ese servlet puede leer. Cambiar Ana por Luis no cambia el servlet; cambia la información recibida.

2 Parámetros en una petición GET

La query string empieza con ?. Cada pareja usa la forma nombre=valor y varias parejas se separan con &. El navegador codifica espacios y caracteres reservados; por ejemplo, un espacio puede aparecer como + o %20. Tomcat descodifica esos datos antes de ofrecerlos mediante request.

```
String nombre = request.getParameter("nombre");
```

getParameter es un método de HttpServletRequest. Tomcat creó request para esta petición concreta y lo entregó al llamar a doGet. El argumento "nombre" es la clave exacta que buscamos. El resultado siempre es un String o null: incluso edad=20 llega inicialmente como "20".

Petición	Resultado de getParameter("nombre")
/saludo?nombre=Ana	"Ana"
/saludo?nombre=Ana%20María	"Ana María"
/saludo?nombre=	""
/saludo	null
/saludo?Nombre=Ana	null

3 Mini práctica 1 Calculadora mediante GET

Objetivo

Crear /calcular?num1=10&num2=5, convertir los dos textos a int y devolver la suma 15.

```
package com.ejemplo.ut3;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet("/calcular")
public class CalcularServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response) throws IOException {
        response.setContentType("text/html;charset=UTF-8");

        String num1Texto = request.getParameter("num1");
        String num2Texto = request.getParameter("num2");

        int num1 = Integer.parseInt(num1Texto);
        int num2 = Integer.parseInt(num2Texto);
        int suma = num1 + num2;

        response.getWriter().println("<h1>Resultado</h1>");
        response.getWriter().println("<p>La suma es " + suma + "</p>");
    }
}
```

Qué hace cada parte importante

- @WebServlet("/calcular") relaciona la ruta con la clase; /ut3-web no se escribe aquí.
- doGet se ejecuta porque la petición enviada desde la barra es GET.
- getParameter busca num1 y num2 dentro de request. Todavía obtiene texto.
- Integer.parseInt recibe un String que debe representar un entero y devuelve un int.
- num1 + num2 realiza suma numérica; con String, el operador + concatenaría.
- getWriter escribe el HTML en el cuerpo de la respuesta, no en la consola.

Ejecutar y reconstruir

1. Despliega el servlet.

2. Abre /ut3-web/calcular?num1=10&num2=5.

3. Comprueba La suma es 15.

4. Cambia los valores y vuelve a probar.

5. Prueba num1=hola y observa el error antes de corregirlo.

```
"10" -- Integer.parseInt --> 10
"hola" -- Integer.parseInt --> NumberFormatException
```

NumberFormatException indica que el texto no tiene un formato entero válido. En este primer código la excepción produce normalmente un error 500. No es un problema de Tomcat ni de la ruta: el servlet se encontró, pero falló al procesar el dato.

4 Parámetros ausentes vacíos y valores por defecto

Caso	Ejemplo	Valor Java
Ausente	/saludo	null: no existe una pareja con esa clave.
Vacío	/saludo?nombre=	"": existe la clave, pero no contiene caracteres.
Espacios	/saludo?nombre=	Una cadena no vacía formada por espacios.
Correcto	/saludo?nombre=Ana	"Ana"

```
if (nombre == null || nombre.isBlank()) {
    nombre = "visitante";
}
```

La comprobación de null va primero porque no se puede llamar a isBlank sobre una referencia null. El operador || deja de evaluar cuando la primera condición ya es verdadera. isEmpty solo comprueba longitud cero. isBlank también considera vacía una cadena formada únicamente por espacios.

Método	""	" "	"Ana"
isEmpty()	true	false	false
isBlank()	true	true	false

5 Mini práctica 2 Saludo validado

```
@WebServlet("/saludo")
public class SaludoValidadoServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response) throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        String nombre = request.getParameter("nombre");

        if (nombre == null) {
            response.setStatus(HttpServletResponse.SC_BAD_REQUEST);
            response.getWriter().println("Falta el parámetro nombre");
        } else if (nombre.isBlank()) {
            response.setStatus(HttpServletResponse.SC_BAD_REQUEST);
            response.getWriter().println("El nombre no puede estar vacío");
        } else {
            response.getWriter().println("<h1>Hola, " + nombre + "</h1>");
        }
    }
}
```

URL	Rama ejecutada	Respuesta
/saludo	nombre == null	400 Falta el parámetro nombre
/saludo?nombre=	nombre.isBlank()	400 El nombre no puede estar vacío
/saludo?nombre=Ana	else	200 Hola, Ana

En los tres casos Tomcat localiza el mismo servlet. Lo que cambia es el contenido de request y, por tanto, la rama elegida. setStatus modifica el código HTTP; el mensaje escrito explica el problema a la persona usuaria.

6 Formularios HTML

Un formulario permite que el navegador construya la petición a partir de controles. No llama a Java directamente. Al pulsar el botón, el navegador recoge los controles que tienen name, forma parámetros y envía una petición a action usando method.

```
<form action="calcular" method="get">
    <label for="num1">Primer número:</label>
    <input id="num1" type="number" name="num1" value="10">
    <button type="submit">Calcular</button>
</form>
```

Elemento	Para qué sirve	Qué llega al servidor
form	Agrupa controles y define destino y método.	No es un parámetro.
action	Ruta que recibirá la petición.	Determina el servlet.
method	GET o POST.	Determina doGet o doPost.
input	Control donde se introduce o selecciona un dato.	Su valor si tiene name.
name	Nombre del parámetro HTTP.	La clave exacta.

Elemento	Para qué sirve	Qué llega al servidor
value	Valor inicial o valor asociado a una opción.	El valor enviado.
label	Texto accesible asociado mediante for e id.	No se envía.
submit	Ordena construir y enviar el formulario.	Provoca la petición.

Conexión fundamental. name="num1" en HTML produce un parámetro num1. Java debe pedir request.getParameter("num1"). id y for mejoran la relación visual y accesible, pero no sustituyen a name.

7 Mini práctica 3 Formulario GET

Crea src/main/webapp/calculadora.html:

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Calculadora</title></head>
<body>
    <h1>Sumar dos números</h1>
    <form action="calcular" method="get">
        <label for="num1">Número 1:</label>
        <input id="num1" name="num1" type="number" required>
        <label for="num2">Número 2:</label>
        <input id="num2" name="num2" type="number" required>
        <button type="submit">Sumar</button>
    </form>
</body>
</html>
```

1. Abre /ut3-web/calculadora.html.

2. Escribe 10 y 5.

3. Pulsa Sumar.

4. Observa /ut3-web/calcular?num1=10&num2=5.

5. Relaciona cada name con el getParameter correspondiente.

```
name="num1"  --> num1=10 --> request.getParameter("num1") --> "10"
name="num2"  --> num2=5  --> request.getParameter("num2") --> "5"
```

El navegador lee los controles, codifica sus valores, construye la query string, envía GET, Tomcat localiza /calcular y llama a doGet. El servlet recupera, convierte, suma y escribe la respuesta.

8 Controles HTML básicos

Control	Ejemplo	Valor recibido
text	<input name="nombre" type="text">	El texto escrito.
number	<input name="edad" type="number">	Texto numérico; Java todavía recibe String.
date	<input name="fecha" type="date">	Normalmente AAAA-MM-DD.
radio	<input name="turno" value="tarde" type="radio">	El value de la única opción marcada.

Control	Ejemplo	Valor recibido
checkbox	<input name="activo" value="si" type="checkbox">	"si" si se marca; null si no se marca.
select	<option value="web">Web</option>	El value de la opción seleccionada.

Los radio que comparten name forman un grupo: solo se selecciona uno. Varios checkbox con el mismo name pueden enviar varios valores; los leeremos con getParameterValues. El texto visible de option no tiene por qué coincidir con el value enviado.

9 Mini práctica 4 Formulario con varios campos

Crea alumno.html con nombre, edad, fecha, turno, activo y ciclo:

```
<form action="resumen-alumno" method="get">
  <label>Nombre: <input name="nombre" type="text" required></label>
  <label>Edad: <input name="edad" type="number" min="16" max="99"></label>
  <label>Fecha: <input name="fecha" type="date"></label>
  <p>Turno:</p>
  <label><input name="turno" type="radio" value="manana"> Mañana</label>
  <label><input name="turno" type="radio" value="tarde"> Tarde</label>
  <label><input name="activo" type="checkbox" value="si"> Matrícula activa</label>
  <select name="ciclo">
    <option value="daw">DAW</option>
    <option value="dam">DAM</option>
    <option value="asir">ASIR</option>
  </select>
  <button type="submit">Enviar</button>
</form>
```

```
String nombre = request.getParameter("nombre");
int edad = Integer.parseInt(request.getParameter("edad"));
String fecha = request.getParameter("fecha");
String turno = request.getParameter("turno");
boolean activo = "si".equals(request.getParameter("activo"));
String ciclo = request.getParameter("ciclo");
```

Integer.parseInt transforma String en int. Double.parseDouble realizará una conversión equivalente para decimales. La expresión "si".equals(...) produce boolean y es segura si el checkbox no está marcado, porque equals se invoca sobre la cadena conocida, no sobre null.

> **Una casilla no marcada no envía su parámetro.** No llega activo="false"; normalmente no llega ninguna clave activo. Por eso getParameter devuelve null.

10 GET frente a POST

Aspecto	GET	POST
Datos habituales	Query string de la URL.	Cuerpo de la petición para formularios normales.
Uso orientativo	Consultar o filtrar.	Enviar datos para crear o modificar.
Marcador y enlace	La URL puede conservar la consulta.	La URL no conserva el cuerpo.
Recarga	Repite una consulta.	Puede intentar reenviar los datos.

Aspecto	GET	POST
Idempotencia introductoria	Repetir debería dejar el mismo estado.	Puede producir un cambio nuevo cada vez.
Privacidad	Los datos quedan visibles en URL e historial.	No aparecen en URL, pero no están cifrados por ello.
Protección de tráfico	HTTPS.	HTTPS.

GET no significa inseguro ni POST seguro. Ambos necesitan HTTPS para cifrar el transporte. La elección expresa la intención de la operación. Una búsqueda encaja en GET; un alta simulada encaja en POST.

11 Mini práctica 5 Convertir GET en POST

```
<form action="resumen-alumno" method="post">
```

```
@WebServlet("/resumen-alumno")
public class ResumenAlumnoServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response) throws IOException {
        request.setCharacterEncoding("UTF-8");
        response.setContentType("text/html;charset=UTF-8");
        String nombre = request.getParameter("nombre");
        String ciclo = request.getParameter("ciclo");
        response.getWriter().println("<h1>Alumno recibido</h1>");
        response.getWriter().println("<p>" + nombre + " - " + ciclo + "</p>");
    }
}
```

Cambia	No cambia
method pasa de get a post.	Los name de los controles.
Tomcat llama a doPost.	El tipo HttpServletRequest.
Los datos viajan normalmente en el cuerpo.	request.getParameter(...).
La URL ya no muestra los parámetros.	La necesidad de validar y configurar UTF-8.

```
Formulario -- POST /resumen-alumno + cuerpo --> Tomcat
Tomcat -- doPost(request,response) --> Servlet
request.getParameter("nombre") --> mismo acceso lógico al dato
```

12 getParameter con POST

request.getParameter no significa leer de la URL. Significa pedir a HttpServletRequest un parámetro ya interpretado. Tomcat puede obtenerlo de la query string o del cuerpo de un formulario application/x-www-form-urlencoded. El servlet utiliza la misma interfaz aunque el transporte cambie.

Orden importante. En POST llama a request.setCharacterEncoding("UTF-8") antes de leer el primer parámetro. Después de leer, la interpretación del cuerpo puede haber ocurrido ya.

13 Parámetros múltiples

```
<p>Lenguajes que conoces:</p>
<label><input type="checkbox" name="lenguajes" value="java"> Java</label>
<label><input type="checkbox" name="lenguajes" value="javascript"> JavaScript</label>
<label><input type="checkbox" name="lenguajes" value="python"> Python</label>
<label><input type="checkbox" name="lenguajes" value="php"> PHP</label>
```

```
String[] lenguajes = request.getParameterValues("lenguajes");
if (lenguajes == null) {
    out.println("<p>No se seleccionó ningún lenguaje</p>");
} else {
    for (String lenguaje : lenguajes) {
        out.println("<li>" + lenguaje + "</li>");
    }
}
```

Selección	Resultado
Solo Java	Array {"java"}
Java y Python	Array {"java", "python"}
Ninguna	null

getParameter devuelve un solo valor. getParameterValues devuelve un array con todos los valores de una clave repetida. Se comprueba null antes del for para evitar un error cuando no se marcó nada.

14 Codificación de caracteres

```
request.setCharacterEncoding("UTF-8");
response.setContentType("text/html;charset=UTF-8");
```

Instrucción	Dirección que protege	Momento
request.setCharacterEncoding	Bytes recibidos → caracteres Java	Antes de leer parámetros del cuerpo.
response.setContentType	Caracteres Java → bytes → navegador	Antes de obtener o usar el escritor.

Sin una codificación coherente, María, Muñoz o programación pueden aparecer con símbolos extraños. El meta charset del HTML ayuda al documento, pero no sustituye la cabecera HTTP de la respuesta ni la codificación de la petición.

15 Construcción de una respuesta HTML

```
PrintWriter out = response.getWriter();
out.println("<!DOCTYPE html>");
out.println("<html lang='es'><head><meta charset='UTF-8'>");
out.println("<title>Datos</title></head><body>");
out.println("<h1>Datos recibidos</h1>");
out.println("<p>Nombre: " + nombre + "</p>");
out.println("</body></html>");
```

PrintWriter es un escritor de caracteres conectado al cuerpo de response. Cada println añade texto a la misma respuesta. Tomcat convierte ese texto a bytes, completa el mensaje HTTP y lo envía. El navegador recibe HTML y lo representa.

Este enfoque permite comprender el flujo, pero mezclar muchas etiquetas y Java se vuelve incómodo cuando la interfaz crece. Más adelante separaremos la vista; todavía no necesitamos JSP ni MVC.

> **Precaución.** Los valores introducidos por usuarios deberían escaparse antes de insertarlos en HTML para impedir inyección. En este bloque practicamos transporte y validación; no reutilices sin protección este patrón con datos no confiables.

16 Validación en servidor

required, min y max mejoran la experiencia en el navegador, pero pueden evitarse modificando la petición. El servidor debe considerar no confiable todo dato recibido y repetir las comprobaciones relevantes.

Regla	HTML	Java en servidor
Obligatorio	required	null o isBlank
Número	type="number"	try con parseInt o parseDouble
Rango	min="1" max="100"	comparar valor < 1 o > 100
Permitido	select con opciones	comprobar contra un conjunto conocido

```
double precio;
try {
    precio = Double.parseDouble(precioTexto);
} catch (NumberFormatException e) {
    errores.add("El precio debe ser numérico");
    precio = 0;
}
if (precio <= 0) errores.add("El precio debe ser mayor que cero");
```

El bloque try intenta convertir. catch se ejecuta si el formato es inválido. Guardar errores permite mostrar varios mensajes en una sola respuesta en vez de fallar con un 500.

17 Mini práctica 6 Alta de producto

Formulario producto.html

```
<form action="alta-producto" method="post">
  <label>Nombre: <input name="nombre" type="text" required></label>
  <label>Precio: <input name="precio" type="number" step="0.01" min="0.01" required></label>
  <select name="categoria">
    <option value="informatica">Informática</option>
    <option value="libros">Libros</option>
    <option value="hogar">Hogar</option>
  </select>
  <label><input name="disponible" type="checkbox" value="si"> Disponible</label>
  <button type="submit">Dar de alta</button>
</form>
```

Servlet completo

```
@WebServlet("/alta-producto")
public class AltaProductoServlet extends HttpServlet {
 @Override
 protected void doPost(HttpServletRequest request,
                       HttpServletResponse response) throws IOException {
  request.setCharacterEncoding("UTF-8");
  response.setContentType("text/html;charset=UTF-8");
  PrintWriter out = response.getWriter();

  String nombre = request.getParameter("nombre");
  String precioTexto = request.getParameter("precio");
  String categoria = request.getParameter("categoria");
  boolean disponible = "si".equals(request.getParameter("disponible"));
  List<String> errores = new ArrayList<>();

  if (nombre == null || nombre.isBlank())
      errores.add("El nombre es obligatorio");

  double precio = 0;
  try {
      precio = Double.parseDouble(precioTexto);
      if (precio <= 0) errores.add("El precio debe ser mayor que cero");
  } catch (NumberFormatException | NullPointerException e) {
      errores.add("El precio debe ser numérico");
  }

  Set<String> categorias = Set.of("informatica", "libros", "hogar");
  if (!categorias.contains(categoria))
      errores.add("La categoría no es válida");

  if (!errores.isEmpty()) {
      response.setStatus(HttpServletResponse.SC_BAD_REQUEST);
      out.println("<h1>Revisa los datos</h1><ul>");
      for (String error : errores) out.println("<li>" + error + "</li>");
      out.println("</ul>");
      return;
  }

  out.println("<h1>Producto recibido</h1>");
  out.println("<p>Nombre: " + nombre + "</p>");
  out.println("<p>Precio: " + precio + "</p>");
  out.println("<p>Categoría: " + categoria + "</p>");
  out.println("<p>Disponible: " + disponible + "</p>");
 }
}
```

Añade los import de java.io.PrintWriter, java.util.ArrayList, java.util.List y java.util.Set. La práctica no guarda el producto: valida y confirma lo recibido. Prueba un nombre vacío, precio texto mediante una petición manipulada, precio cero, categoría inventada y un caso correcto.

18 Errores frecuentes

Síntoma	Causa probable	Comprobación	Solución
getParameter devuelve null	name distinto o control no enviado	Compara HTML y Java; mira la petición	Iguala las claves y contempla checkbox no marcado
NumberFormatException	Texto vacío o no numérico	Imprime o inspecciona el texto antes de convertir	Valida y usa try catch

Síntoma	Causa probable	Comprobación	Solución
404	action, context path o ruta incorrectos	Divide la URL en sus partes	Corrige action o @WebServlet y redespliega
405	Método no implementado	Compara method con doGet/doPost	Implementa el método correcto
Acentos extraños	UTF-8 tarde o ausente	Prueba María y Muñoz	Configura request y response antes de leer/escribir
No conecta	Tomcat parado o puerto incorrecto	Revisa estado y registro	Arranca Tomcat y usa su puerto
Checkbox null	No estaba marcado	Observa que no se envía	Trátalo como false
Conversión errónea	Tipo Java incompatible	Revisa valor real recibido	Convierte tras validar

19 Práctica final guiada Inscripción

Objetivo y estructura

Construir una aplicación sin base de datos con una consulta GET /buscar?nombre=... y un alta simulada POST /inscribir. Archivos: BuscarServlet.java, InscribirServlet.java, buscar.html e inscripcion.html.

Consulta GET

```
<form action="buscar" method="get">
  <label>Nombre: <input name="nombre" type="text" required></label>
  <button type="submit">Buscar</button>
</form>

@WebServlet("/buscar")
public class BuscarServlet extends HttpServlet {
 protected void doGet(HttpServletRequest request,
                      HttpServletResponse response) throws IOException {
  response.setContentType("text/html;charset=UTF-8");
  String nombre = request.getParameter("nombre");
  if (nombre == null || nombre.isBlank()) {
   response.setStatus(400);
   response.getWriter().println("Indica un nombre");
   return;
  }
  response.getWriter().println("<h1>Consulta simulada</h1>");
  response.getWriter().println("<p>Se buscaría a: " + nombre + "</p>");
 }
}
```

Alta simulada POST

```
<form action="inscribir" method="post">
 <label>Nombre: <input name="nombre" required></label>
 <label>Edad: <input name="edad" type="number" min="16" max="99" required></label>
 <select name="turno"><option value="manana">Mañana</option><option
value="tarde">Tarde</option></select>
 <label><input type="checkbox" name="lenguajes" value="java">Java</label>
 <label><input type="checkbox" name="lenguajes" value="python">Python</label>
 <button type="submit">Inscribir</button>
</form>
```

```
@WebServlet("/inscribir")
public class InscribirServlet extends HttpServlet {
 protected void doPost(HttpServletRequest request,
                       HttpServletResponse response) throws IOException {
  request.setCharacterEncoding("UTF-8");
  response.setContentType("text/html;charset=UTF-8");
  String nombre=request.getParameter("nombre");
  String edadTexto=request.getParameter("edad");
  String turno=request.getParameter("turno");
  String[] lenguajes=request.getParameterValues("lenguajes");
  List<String> errores=new ArrayList<>();
  if(nombre==null || nombre.isBlank()) errores.add("Nombre obligatorio");
  int edad=0;
  try { edad=Integer.parseInt(edadTexto); if(edad<16 || edad>99) errores.add("Edad entre 16 y
99"); }
  catch(Exception e){ errores.add("Edad no válida"); }
  if(!Set.of("manana","tarde").contains(turno)) errores.add("Turno no válido");
  PrintWriter out=response.getWriter();
  if(!errores.isEmpty()){
   response.setStatus(400); out.println("<h1>Errores</h1><ul>");
   for(String error:errores) out.println("<li>"+error+"</li>");
   out.println("</ul>"); return;
  }
  out.println("<h1>Inscripción recibida</h1>");
  out.println("<p>"+nombre+", "+edad+", turno "+turno+"</p>");
  out.println("<p>Lenguajes: "+(lenguajes==null ? "ninguno" : String.join(",
",lenguajes))+"</p>");
 }
}
```

Pruebas obligatorias

- Buscar Ana y comprobar que la URL conserva ?nombre=Ana.
- Enviar una inscripción válida y comprobar que la URL no contiene los campos.
- Enviar edad 15, edad vacía y edad no numérica mediante una petición manipulada.
- No marcar lenguajes y comprobar que no falla.
- Probar un nombre con tilde y una eñe.
- Explicar el recorrido completo de un GET y de un POST.

```
Usuario -> formulario -> petición HTTP -> Tomcat -> servlet
-> request -> validación Java -> response -> navegador
```

20 Actividades de autonomía creciente

1. Añade un campo correo y valida que no esté vacío.

2. Añade un radio modalidad con presencial y distancia; rechaza cualquier otro valor en Java.

3. Añade JavaScript y PHP al grupo lenguajes sin cambiar la lógica de getParameterValues.

4. Convierte la búsqueda GET en POST y anota qué ventajas de la URL se pierden.

5. Personaliza el resumen final para indicar cuántos lenguajes se seleccionaron.

En cada modificación escribe antes: qué name viajará, qué método HTTP se enviará, qué método del servlet se ejecutará y qué valor puede devolver request.

21 Mapa mental y cierre

```
USUARIO
  | escribe URL o completa controles
  v
FORMULARIO / URL
  | name + value forman parámetros
  v
PETICIÓN HTTP  GET: query string | POST: normalmente cuerpo
  v
TOMCAT  selecciona aplicación, ruta y doGet/doPost
  v
HttpServletRequest
  | getParameter / getParameterValues
  v
PROCESAMIENTO JAVA  comprobar, convertir, calcular
  v
HttpServletResponse  estado + tipo + cuerpo
  v
RESPUESTA HTTP
  v
NAVEGADOR  interpreta y muestra
```

Ideas que deben quedar claras

- La ruta selecciona el servlet; los parámetros transportan datos.
- name en HTML es la clave que Java recupera.
- getParameter devuelve String o null tanto con GET como con POST.
- Ausente, vacío y espacios son casos diferentes.
- Toda conversión puede fallar y debe validarse.
- Un checkbox no marcado normalmente no envía parámetro; varios valores requieren getParameterValues.
- GET sirve para consultas repetibles y enlazables; POST suele expresar procesamiento o cambio.
- POST no cifra: la protección del transporte depende de HTTPS.
- La validación del navegador ayuda, pero la decisión fiable se toma en el servidor.
- request se configura antes de leer y response antes de escribir.

Cuando puedas narrar este recorrido sin depender del código, estarás preparado para separar responsabilidades y construir interfaces más mantenibles en los siguientes bloques.